

Computer Security Exam

Professors M. Carminati & S. Zanero

09/09/2021

Last (family) Name _____

First (given) Name _____

Matricola _____

Codice Persona _____

Professor: ☐ Carminati ☐ Zanero

Have you done any challenges/homework, even partially? ☐ Yes ☐ No

Are you going to graduate in the current call? [Priority] ☐ Yes ☐ No

Do you want to withdraw ? ☐ Yes ☐ No

Instructions

- **Duration:** 2 hours.
- The exam is composed of 18 pages. Check that you have all of them.
- Just as a cross check, tell us whether you have completed the homeworks, by putting an "X" mark appropriately.
- The exam is "open book". The students will be allowed to consult only the **printed course material** and **notes**. **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, if you do not follow this rule.
- Shut down and store electronic devices. They will be subject to inspection if found and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good, short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**

- You will be allowed to consult only printed documents or notes as supporting materials. No digital devices (other than the computer you are reading the exam from) will be allowed.
- On the computer, you will be allowed to open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
- You will have to share your screen, leave your microphone opened, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct delivery of the exam.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or for a subset, at the instructor's decision.
- Regarding the submission :
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents
 - We will evaluate only the uploaded **readable** documents
 - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf)

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 (10 points)

Assume that:

- The C standard library is loaded at a known address during every execution of the program, and that the address of the function `system()` is `0xf4d0e2d3`.
- Environment variables are located in the highest addresses.
- *The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdec1** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors is present (unless specified otherwise).*

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <string.h>
04 #include <stdint.h>
05 #include <time.h>
06
07 typedef struct {
08     char user[8];
09     char secret[8];
10     char message[64];
11 } data_t;
12
13 void load_secret(data_t *data) {
14     int *fd;
15
16     fd = fopen("secret.txt", "r");
17     fscanf(fd, "%[^\\n]", data->secret);
18     fclose(fd);
19     data->secret[7]='\0';
20 }
21
22 void random_stuff(data_t *data) {
23     char buf[16];
24
25     snprintf(data->message, 64, "Welcome user : %s\\n", data->user);
26     printf(data->message);
27     scanf("%s", buf);
28
29     if (strncmp(buf, "ST4RT", 4) == 0 && strlen(buf) > 12){
30         printf("%d", strlen(buf));
31     } else {
32         abort();
33     }
34
35
36 void main(int argc, char** argv) {
37     data_t data;
38
39     scanf("%7s", data.user);
40     load_secret(&data);
41     random_stuff(&data);
42
43     if (strcmp(data.secret, data.user) == 0)
44         system("/bin/sh\\0");
45 }
```

[1 point] 1. The program is affected by a typical buffer overflow and format string vulnerability. Complete the following table, focusing on a vulnerability per row.

Vulnerability	Line	<u>Modify the line to solve the detected vulnerability</u>
Buffer Overflow		
Format String		

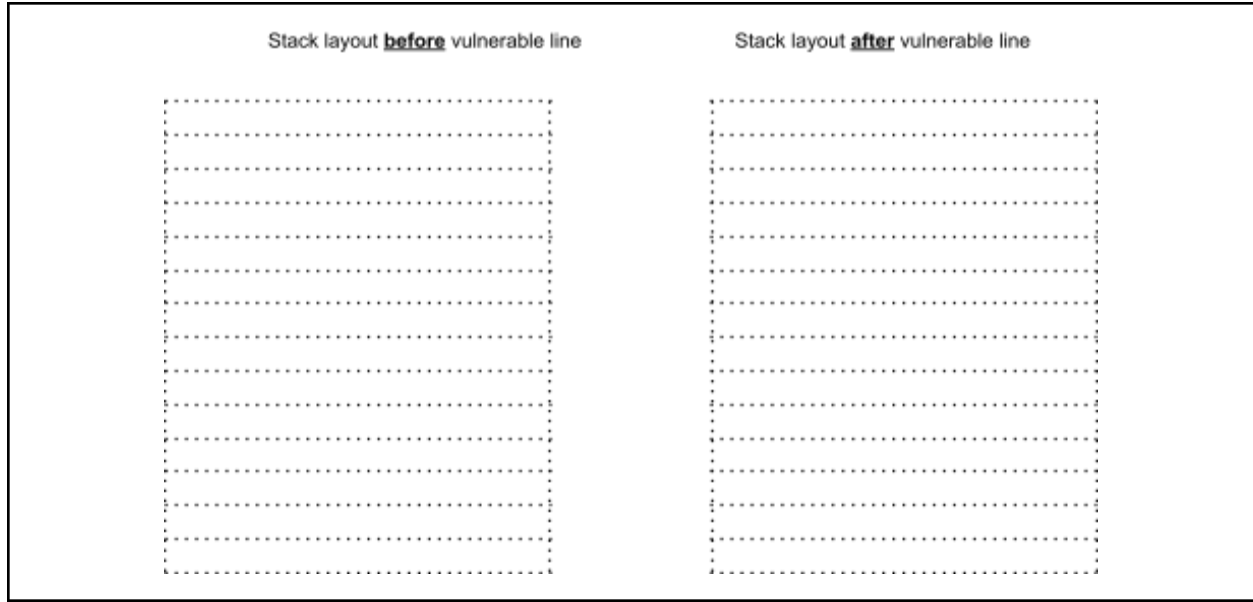
[1 point] 2. Draw the stack layout after the program has executed the instruction at line 15, showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

[1 point] 3. Focus only on the stack-based buffer overflow(s) you found. Write an exploit for this vulnerability that **must execute the following shellcode**, composed of 8 bytes, which opens a shell: **0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90**.

Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation. Ensure you describe **all the steps and assumptions** required for a successful exploitation of the vulnerability. Include also any assumption on how you must call the program (e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables, if any).



[1 point] 4. Focus only on the format string vulnerability you found. Write an exploit that executes the previous shellcode, assuming that you have already prepared the memory with the correct arguments. Assume that:

- The address of the return address (*saved EIP*) of the function exploited is: **0xe5a0fe4b** (i.e., where to write)
- The address of the first instruction of the **shellcode** is in an **environment variable** at: **0xcafebabe** (i.e., what to write)
- The displacement on the stack of the vulnerable function's argument is: **4**

Knowing that $dec(0xcafe) = 51966$ and $dec(0xbabe) = 47806$, write the exploit clearly, detailing all the components of the format string and all the steps that lead to a successful exploitation.

From now on assume that the “Non-executable stack (NX)” mitigation technique is correctly implemented and enabled.

[1 point] 5. **Focus only on the stack-based buffer overflow(s) you found.** Is the implemented mitigation effective to prevent **your exploits** at point 3.? If the mitigation technique is effective, explain why and describe how you would modify the exploits to bypass the mitigation. Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation. Ensure you describe **all the steps and assumptions** required for a successful exploitation of the vulnerability. Include also any assumption on how you must call the program. If it is not effective, please explain why.

Stack layout <u>before</u> vulnerable line	Stack layout <u>after</u> vulnerable line

[1 point] 6. Is the implemented mitigation effective to prevent **your exploits** at point 4.? If the mitigation technique is effective, explain why and describe how you would modify the exploits to bypass the mitigation. Assume that you have already prepared the memory with the correct arguments. Ensure you describe **all the assumptions** required for a successful exploitation of the vulnerability and how the address of “where to write”, “what to write”, and the “displacement on the stack” would change w.r.t. the previous exploit. If it is not effective, please explain why.

[1 point] 7. A friend of yours suggests that there is -- at least -- an alternative way (different from the previous ones) **to exploit the vulnerabilities you found** to spawn a privileged shell in the program **without** executing any shellcode or calling library functions. If it is possible, please explain **all the steps and assumptions** required for a successful exploitation of the vulnerability. Include also any assumption on how you must call the program (e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables, if any). If it is not effective, please explain why.

From now on assume that the “Non-executable stack (NX)” and the “Stack Canary” mitigation techniques are correctly implemented and enabled.

[2 points] 8. Do the exploits you wrote at previous points **5.** and **6.** work without any modification? If the answer is not, explain why and describe how you would exploit the vulnerabilities you found in the program above to achieve the same goal (i.e., spawn a privileged shell). If you need to exploit multiple vulnerabilities, clearly explain **all the steps and assumptions** required for a successful exploitation. If the answer is yes, please explain why.

[1 point] 7. Consider remote exploitation (i.e., you do not have local access to the machine on which the vulnerable program is executed, not SSH, you can interact only with inputs and outputs). Which of the previous exploits (3. 4. 5. 6. 7. 8.) would work with minor modifications? Motivate your answer also providing the assumption needed for a successful exploitation. Assume the same mitigations that were active in the point under analysis.

Exercise 2 (9 points)

OpenHours is a great gym that provides a unique and cool service: it helps you monitor your progress without the need for a personal trainer. Everytime you go to OpenHours, you measure your weight, height, percentage of body fat and muscles and much more other info. Every time you do an exercise, you place your NFC card on the machine and it will upload your performance on OpenHours's servers. In this way, you can monitor your progress over time and see which exercises are more effective to you. When you join OpenHours, you are given access to an account that you can use on OpenHours's webapp to access those data. Cool, isn't it?

Since OpenHours is becoming big they have decided to hire new security consultants to make sure their service doesn't leak important data. You are one of those! You are given some relevant piece of code to analyze and you are asked to answer the following questions.

```
# POST request at https://openhours.com/login
35 def login(request):
36     if not is_session_valid(request) or request.method != "POST":
37         abort(403)
38     email = request.form.get('email')
39     password = request.form.get('password')
40
41     query_statement = "SELECT * FROM WEALTH WHERE email = '" + email + "'"
and password = "'" + password + "'"
42     results = db_execute_query(query_statement)
43
44     if len(results) < 1:
45         return render_page('login.html', "Incorrect username or
password")
46
47     session['email'] = email
50     return render_page('welcome.html', email)

# GET request at https://openhours.com/wealth
50 def show_personal_wealth(request):
51     if not is_session_valid(request) or request.method != "GET":
52         abort(403)
53     email = check_session(request.cookies['session'])
54     day = request.args.get("day")
55
56     query_statement = "SELECT * FROM WEALTH WHERE email = '" + email + "'"
and training_day = "'" + day + "'"
57     results = db_execute_query(query_statement)
58     return render_page('personal_wealth.html', results)
```

The interesting tables of the database are the following:

USERS

email (string - primary key)	password_hash (string)
------------------------------	------------------------

WEALTH

training_day (composite primary key, text)	email (composite primary key, also foreign referencing USERS.username, text)	weight (double)	height (double)	muscle_perc (double)
---	---	--------------------	--------------------	-------------------------

Assume the following:

- The object session and the function check_session() securely manages the users' sessions.
- The library used to access the DBMS does not support stacked queries, i.e., the DBMS does not execute multiple queries if asked on the same query and separated by a semicolon. For instance, the query "select x from y where a=b; update k set [...]" would trigger an error and not be executed.
- The function db_execute_query() takes a query and executes it on the database.
- The function render_page() takes a template of an html page and some extra parameters and it accurately places them in the webpage, performing adequate *html escaping*.

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, **be precise, specify all the assumptions, and most importantly be specific to this particular exercise**. If you believe there might be a vulnerability, specify the corresponding line/s of code.

Vulnerability class	Is there a vulnerability of this class in the code above? Which lines cause it? How could an adversary exploit it?	If the vulnerability is present, explain the simplest procedure to remove it.
cross-site scripting (XSS) Specify if reflected, stored, DOM...		
Cross site request forgery (CSRF)		

Sql Injection (SQL-I)		.
--------------------------	--	---

2. [2 points] Specify how to perform an attack to leak the whole USERS table, assuming you don't know any username that you haven't registered yourself.

--

3. [1 point] Can you obtain the cleartext of the password from the obtained hashes? Discuss the simplest attack and its mitigation.

--

4. [1 point] Assume that you are logged in with your personal account and, for this question only, that queries can be concatenated (e.g., query 1 ; query 2;). Write an exploit so that you can make the browser of "shang.chi@polimi.it" display the string "Don't panic". Please specify all the steps and assumptions you need to make for a successful exploitation, including whether you need to assume interaction with the victim or not.

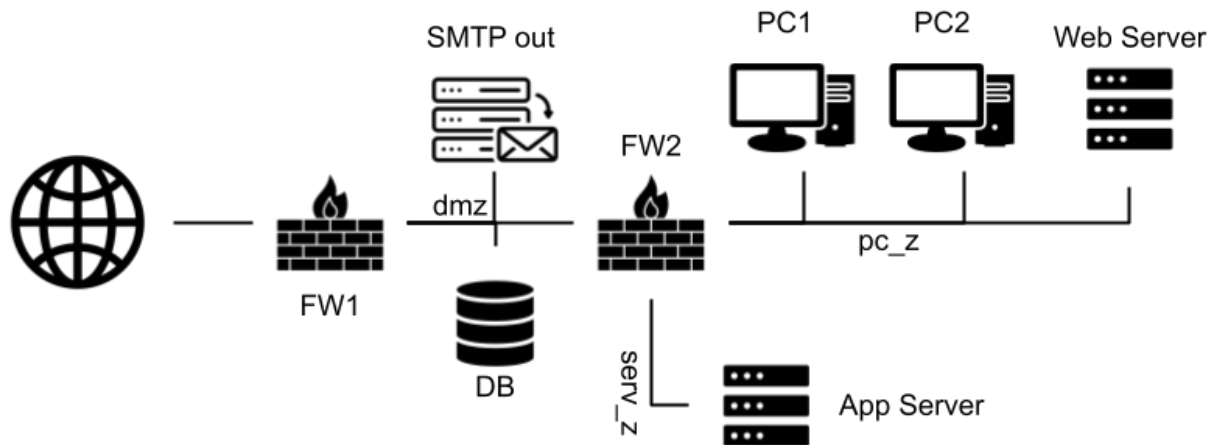
5. [2 points] Assume that you are in the same network as Martina, who is going to visit her personal page on the website. Remember that the website is using https and it's available at <https://openhours.com>. To make the page loading process faster, the website loads all the necessary javascript from a third party CDN. This is done by adding the following line to the HTML:

`<SCRIPT SRC="http://cdn.provider.com/XPqhCdZR0185NS/js/code.js">`

Is there a way to leak what Martina will see on her personal page? If so, carefully describe how to carry on the attack and what the owner of the website could do to mitigate it. Otherwise, explain why that's not possible.

Exercise 3 (9 points)

In the image below there's the representation of the local network of the Company TVA. The PCs require full internet access, while the Web Server only requires to be reachable from the Internet through HTTP/HTTPS. The Application Server requires communication to the Database and to the SMTP Server. The Web Server requires access to the Application Server on custom port 9999. Finally, the SMTP Server requires access to the internet on ports 25 and 587. Assume that each device connection is **switched**.



1. [2 points] However, it is evident that the network is not correctly designed. Represent a modified network that satisfies the previous requirements **while preserving the same zones and firewalls**. Briefly justify each of the changes that you'd implement explaining why the current configuration is not secure.

2. [2 points] Write the firewall rules **of the new network you designed**, assuming firewalls to be stateful packet filters (i.e. you can consider the response rules implicit).

Firewall 1:

Src IP	Src PORT	Direction of the 1st packet	Dst IP	Dst PORT	Policy	Description
10.0.0.1 (example)	ANY	zone 1 -> zone 2	192.168.0.2 (example)	443	DENY	(example: the X server in zone 1 cannot contact the Y server)

Firewall 2:

Src IP	Src PORT	Direction of the 1st packet	Dst IP	Dst PORT	Policy	Description
10.0.0.1 (example)	ANY	zone 1 -> zone 2	192.168.0.2 (example)	443	DENY	(example: the X server in zone 1 cannot contact the Y server)

You notice suspicious traffic on the local network of the company, specifically in the **pc_z** zone. Deciding to investigate, you remotely connect as an administrator to the Ethernet switch that connects the nodes in **pc_z**. Below you can find the excerpt of the network traffic as logged by the switch. Moreover, you are aware that only one device is connected to each of the switch ports, and the last column, **switch direction**, shows from where to where the packet is being forwarded by the switch.

Finally, you are aware of the MAC addresses and Switch Ports of the devices in **pc_z**, which are **highlighted in the log**:

FW2 (Gateway): 43:4a:f4:d7:c6:c2, Port 3

PC1: 8e:34:65:2d:08:22, Port 1

PC2: fc:59:ad:75:10:26, Port 2

Web Server: 4d:d3:fc:56:da:6a, Port 4

	Source MAC	Src. IP	Destination MAC	Dest. IP	Type	Switch direction
1	71:b5:3c:b7:fc:99	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Port 3
2	0d:6c:9b:ed:41:89	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Port 3
3	3f:a9:eb:88:57:be	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Port 3
4	fc:68:d1:7a:9f:8d	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Port 3
5	5c:ec:3f:67:ed:60	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Port 3
6	51:0c:2e:23:ab:0e	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Port 3
7	c0:35:ee:63:96:60	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Port 3
8	fa:16:e1:05:ee:37	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Port 3
9	42:bc:24:97:ae:fe	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Port 3
..	[...]					
111	37:36:fc:07:de:9a	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Broadcast
112	dc:ea:16:34:bf:95	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Broadcast
113	fc:59:ad:75:10:26	192.168.0.20	00:00:00:00:00:00	192.168.1.1	ARP REQ. Who is GW?	Port 2-> Broadcast
114	b6:f2:36:8d:f2:8e	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Broadcast
115	b2:3f:2b:e1:44:3b	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Broadcast
116	bc:c9:cf:2d:64:0b	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Broadcast
117	69:3f:9d:7e:5a:51	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Broadcast
118	df:75:7d:aa:b1:e7	192.168.0.1	fc:59:ad:75:10:26	192.168.0.20	ARP RESP. I am GW	Port 1 -> Broadcast
119	f9:e5:e5:cc:ef:1a	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Broadcast
120	43:4a:f4:d7:c6:c2	192.168.0.1	fc:59:ad:75:10:26	192.168.0.20	ARP RESP. I am GW	Port 3 -> Broadcast
121	3e:6e:de:35:59:22	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 2 -> Broadcast
122	08:7d:76:0e:38:21	192.168.1.34	5f:7d:91:bb:b4:df	192.168.1.1	Generic	Port 1 -> Broadcast

3. [3 points] Describe which kind of attacks are taking place, explaining **(a)** the attack names **(b)** a brief description of how the attacks work in general **(c)** the attacker goals **(d)** a brief description of how you detected the attacks from the log.

4. [1 point] Can you tell something about the attacker's location, MAC, and IP addresses? Motivate your answer.

5. [1 point] It is clear that the attacker is exploiting the first attack to put in place the second. Is it necessary to do so, or would the attacker be able to put in place the second attack without implementing the first? Motivate your answer.

Exercise 4 (6 points)

Our systems have been compromised by a very powerful malware. Luckily, we succeeded in collecting a sample of the malware. Its code is reported below.

Sample	
0x08048040<init_routine>:	
0x08048000 <+0>:	push ebp
0x08048001 <+1>:	mov ebp, esp
0x08048003 <+3>:	push 0x1000
0x08048008 <+8>:	call 0x8047000 <sleep@plt> ¹
0x0804800d <+13>:	add esp, 0x4
0x08048010 <+16>:	mov eax, 0x1
0x08048015 <+21>:	cpuid ²
0x08048017 <+23>:	and ecx, 0x80000000
0x0804801d <+29>:	cmp ecx, 0x0
0x08048020 <+32>:	jnz 0x08048027 <init_routine + 39>
0x08048022 <+34>:	call 0x8049000 <evil>
0x08048027 <+39>:	pop ebp
0x08048028 <+40>:	ret

- ¹sleep() causes the calling thread to sleep either until the number of real-time seconds specified in seconds have elapsed or until a signal arrives which is not ignored.
- ²Excerpt from the Intel IA-32 manual: “CPUID returns processor identification and feature information in the EAX, EBX, ECX, and EDX registers. The instruction’s output is dependent on the contents of the EAX register upon execution (in some cases, ECX as well).” In particular, the 31st bit of ecx(MSB) indicates if the program is running in a virtual environment or not [1 = Virtual Environment - 0 Not Virtual Environment].

[2 point] It is clear that the malware is showing evasive behavior. What technique(s) is(are) implemented? Explain how it works in this specific case.

[2 point] Would you perform static or dynamic analysis to analyze and detect this malware? Please, describe in detail how you would perform the analysis and the detection of this malware.

[2 point] We collected another sample which seems related to the previous one. However, this new sample employs polymorphism to encrypt the payload (possibly the `init_routine` function and all the functions called by it). You want to analyze the sample to understand if the behaviour is the same of the first sample you retrieve. Which kind of analysis would you perform to verify if the two samples have the same behaviour? Is the detection techniques employed in the previous point still valid?